

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5

8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I Sandra Sudevan(192424422) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sandra sudevan

1. Heap File Organization

Aim

To simulate a heap file organization and perform **insert, delete, and sequential scan** operations.

Procedure

1. Create a database and Student table.
2. Insert student records.
3. Display all records using SELECT.
4. Delete a record using its ID.
5. Perform sequential scanning again.

```
mysql>
mysql> -- Sequential Scan
Query OK, 0 rows affected (0.009 sec)

mysql> SELECT * FROM Student;
+-----+-----+-----+
| ID    | Name  | Mark  |
+-----+-----+-----+
| 1     | Arun  | 85    |
| 2     | Priya | 90    |
| 3     | Kavın | 78    |
+-----+-----+-----+
3 rows in set (0.014 sec)

mysql>
mysql> -- Delete
Query OK, 0 rows affected (0.002 sec)

mysql> DELETE FROM Student
      -> WHERE ID = 2;
Query OK, 1 row affected (0.090 sec)

mysql>
mysql> -- Sequential Scan after deletion
Query OK, 0 rows affected (0.003 sec)

mysql> SELECT * FROM Student;
+-----+-----+-----+
| ID    | Name  | Mark  |
+-----+-----+-----+
| 1     | Arun  | 85    |
| 3     | Kavın | 78    |
+-----+-----+-----+
2 rows in set (0.009 sec)
```

Result

The heap file organization was successfully simulated using insert, delete, and sequential scan operations.

2. Static Hashing with Chaining

Aim

To implement a static hashing technique for student records and handle collisions using chaining.

Procedure

1. Create a student table.
2. Use MOD() as the hash function.
3. Calculate the bucket number from the student ID.
4. Store records in corresponding buckets.
5. Display the hash table.

```
mysql> CREATE TABLE Hash_Student (
->     ID INT,
->     Name VARCHAR(30),
->     Bucket INT
-> );
Query OK, 0 rows affected (0.740 sec)

mysql>
mysql> INSERT INTO Hash_Student VALUES
-> (101, 'Arun', MOD(101, 5)),
-> (102, 'Priya', MOD(102, 5)),
-> (106, 'Kavin', MOD(106, 5)),
-> (111, 'Divya', MOD(111, 5)),
-> (116, 'Rahul', MOD(116, 5));
Query OK, 5 rows affected (0.192 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Hash_Student
-> ORDER BY Bucket;
+-----+-----+-----+
| ID    | Name  | Bucket |
+-----+-----+-----+
| 101   | Arun  | 1      |
| 106   | Kavin | 1      |
| 111   | Divya | 1      |
| 116   | Rahul | 1      |
| 102   | Priya | 2      |
```

Result

Static hashing was successfully implemented using MOD(), and records having the same bucket value demonstrate collision handling.

3. Sorted File Organization and Binary Search

Aim

To implement sorted file organization and retrieve records based on a primary key.

Procedure

1. Create a Student table.
2. Insert records with different IDs.
3. Display records in sorted order.
4. Search for a particular primary key.
5. Use the primary key index for efficient retrieval.

```
mysql>
mysql> INSERT INTO Sorted_Student VALUES
    -> (105, 'Kavin', 80),
    -> (101, 'Arun', 90),
    -> (110, 'Priya', 85),
    -> (103, 'Divya', 88),
    -> (108, 'Rahul', 75);
Query OK, 5 rows affected (0.115 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Sorted display
Query OK, 0 rows affected (0.002 sec)

mysql> SELECT * FROM Sorted_Student
    -> ORDER BY ID;
+-----+-----+-----+
| ID | Name | Mark |
+-----+-----+-----+
| 101 | Arun | 90 |
| 103 | Divya | 88 |
| 105 | Kavin | 80 |
| 108 | Rahul | 75 |
| 110 | Priya | 85 |
+-----+-----+-----+
5 rows in set (0.005 sec)

mysql>
mysql> -- Search
Query OK, 0 rows affected (0.006 sec)

mysql> SELECT * FROM Sorted_Student
    -> WHERE ID = 108;
+-----+-----+-----+
| ID | Name | Mark |
+-----+-----+-----+
| 108 | Rahul | 75 |
+-----+-----+-----+
1 row in set (0.223 sec)
```

Result

The records were successfully displayed in sorted order and retrieved using the primary key.

4. B-Tree of Order 4

Aim

To demonstrate indexing using a B-Tree structure with the keys 10, 20, 5, 6, 12, 30, 7, 17.

Procedure

1. Create a table containing the required keys.
2. Create an index on the key column.
3. Insert the given keys.
4. Display the keys in sorted order.
5. Search for a particular key.
6. MySQL internally uses a B-Tree/B+Tree-style index for suitable indexes.

```
mysql>
mysql> -- Search
Query OK, 0 rows affected (0.003 sec)

mysql> SELECT * FROM BTree_Data
    -> WHERE KeyValue = 17;
+-----+
| KeyValue |
+-----+
|      17 |
+-----+
1 row in set (0.017 sec)

mysql>
mysql> -- Show index
Query OK, 0 rows affected (0.002 sec)

mysql> SHOW INDEX FROM BTree_Data;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comme |
| Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| btree_data | 1 | idx_btree | 1 | KeyValue | A | 8 | NULL | NULL | YES | BTREE | | |
| YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (1.142 sec)
```

Result

The given keys were successfully inserted, sorted, searched, and indexed using a MySQL index.

5. B+ Tree and Leaf-Level Linking

Aim

To demonstrate B+ Tree indexing and efficient range retrieval using an indexed integer column.

Procedure

1. Create a table containing integer keys.
2. Create an index on the key column.
3. Insert records.
4. Perform a range query.
5. Use EXPLAIN to observe index-based retrieval.

```
mysql>
mysql> -- Range Query
Query OK, 0 rows affected (0.002 sec)

mysql> SELECT *
  -> FROM BPlus_Data
  -> WHERE ID BETWEEN 15 AND 45;
+-----+
| ID | Name |
+-----+
| 15 | Priya |
| 20 | Kavin |
| 25 | Divya |
| 30 | Rahul |
| 35 | Anu |
| 40 | Vijay |
| 45 | Meena |
+-----+
7 rows in set (0.013 sec)

mysql>
mysql> -- Check index usage
Query OK, 0 rows affected (0.002 sec)

mysql> EXPLAIN
  -> SELECT *
  -> FROM BPlus_Data
  -> WHERE ID BETWEEN 15 AND 45;
+-----+
| EXPLAIN
|
+-----+
| -> Filter: (bplus_data.ID between 15 and 45) (cost=1.66 rows=7)
|   -> Index range scan on BPlus_Data using PRIMARY over (15 <= ID <= 45) (cost=1.66 rows=7)
|
+-----+
1 row in set (0.258 sec)
```

Result

The B+ Tree-style index was successfully created and used for efficient range retrieval.

6. Dense Primary Index

Aim

To create a **dense primary index** on a sorted file and retrieve records using the index key.

Procedure

1. Create a Student table.
2. Define the primary key.
3. Insert sorted records.
4. The primary key creates an index entry for each record.
5. Search using the primary key.
6. Display the index using SHOW INDEX.

```
+-----+-----+-----+
| ID | Name | Mark |
+-----+-----+-----+
| 101 | Arun | 90 |
| 105 | Priya | 85 |
| 110 | Kavin | 88 |
| 115 | Divya | 92 |
| 120 | Rahul | 80 |
+-----+-----+-----+
5 rows in set (0.005 sec)

mysql>
mysql> -- Search using index key
Query OK, 0 rows affected (0.005 sec)

mysql> SELECT *
-> FROM Dense_Student
-> WHERE ID = 110;
+-----+-----+-----+
| ID | Name | Mark |
+-----+-----+-----+
| 110 | Kavin | 88 |
+-----+-----+-----+
1 row in set (0.007 sec)

mysql>
mysql> -- Display index
Query OK, 0 rows affected (0.003 sec)

mysql> SHOW INDEX FROM Dense_Student;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_com |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| dense_student | 0 | PRIMARY | 1 | ID | A | 5 | NULL | NULL | NULL | BTREE | | |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.263 sec)
```

Result

A primary index was successfully created and used to retrieve records efficiently.

7. Extendible Hashing

Aim

To demonstrate the concept of **extendible hashing and directory expansion** using SQL.

Procedure

1. Create a table to store records.
2. Calculate the initial bucket using a hash function.
3. Insert records into buckets.
4. Detects bucket overflow.
5. Increase the number of buckets when required.
6. Redistribute records.


```

mysql>
mysql> -- Directory doubling: use 4 buckets
Query OK, 0 rows affected (0.003 sec)

mysql> UPDATE Extendible_Hash
    -> SET Bucket = MOD(ID, 4);
Query OK, 2 rows affected (0.076 sec)
Rows matched: 4  Changed: 2  Warnings: 0

mysql>
mysql> -- Insert additional records
Query OK, 0 rows affected (0.006 sec)

mysql> INSERT INTO Extendible_Hash VALUES
    -> (5, 'Rahul', MOD(5, 4)),
    -> (6, 'Anu', MOD(6, 4)),
    -> (7, 'Vijay', MOD(7, 4)),
    -> (8, 'Meena', MOD(8, 4));
Query OK, 4 rows affected (0.079 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Extendible_Hash
    -> ORDER BY Bucket;
+-----+-----+-----+
| ID    | Name  | Bucket |
+-----+-----+-----+
| 4     | Divya | 0      |
| 8     | Meena | 0      |
| 1     | Arun  | 1      |
| 5     | Rahul | 1      |
| 2     | Priya | 2      |
| 6     | Anu   | 2      |
| 3     | Kavın | 3      |
| 7     | Vijay | 3      |
+-----+-----+-----+
8 rows in set (0.007 sec)

```

Result

The concept of extendible hashing was successfully demonstrated by increasing the number of buckets from 2 to 4 and redistributing records.

8. Sequential vs Indexed Retrieval

Aim

To compare sequential and indexed retrieval by observing the query execution plan and I/O-related access cost.

Procedure

1. Create a table with records.

2. Insert multiple records.
3. Execute a sequential query without an index.
4. Create an index on the search column.
5. Execute the same query.
6. Use EXPLAIN to compare the access methods.

```
mysql> EXPLAIN
-> SELECT * FROM Employee
-> WHERE ID = 4;
+-----+-----+
| EXPLAIN |
+-----+-----+
| -> Filter: (employee.ID = 4) (cost=0.75 rows=1)
  -> Table scan on Employee (cost=0.75 rows=5)
|
+-----+-----+
1 row in set (0.015 sec)

mysql>
mysql> -- Create index
Query OK, 0 rows affected (0.002 sec)

mysql> CREATE INDEX idx_employee
-> ON Employee(ID);
Query OK, 0 rows affected (0.592 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Indexed retrieval
Query OK, 0 rows affected (0.003 sec)

mysql> EXPLAIN
-> SELECT * FROM Employee
-> WHERE ID = 4;
+-----+-----+
| EXPLAIN |
+-----+-----+
| -> Index lookup on Employee using idx_employee (ID = 4) (cost=0.35 rows=1)
|
+-----+-----+
1 row in set (0.016 sec)
```

Result

The execution plan was successfully used to compare sequential and indexed retrieval. Indexed retrieval can reduce the amount of data that must be examined.

9. Sparse Index

Aim

To demonstrate a **sparse index** on a sorted file and compare it conceptually with a dense index.

Procedure

1. Create a sorted student table.
2. Insert records in primary-key order.
3. Create an index on the primary key.
4. Retrieve records using the index.
5. Compare the number of index entries conceptually with a dense index.

```

mysql>
mysql> -- Search
Query OK, 0 rows affected (0.006 sec)

mysql> SELECT *
      -> FROM Sparse_Student
      -> WHERE ID = 106;
+-----+-----+-----+
| ID | Name | Mark |
+-----+-----+-----+
| 106 | Anu | 87 |
+-----+-----+-----+
1 row in set (0.006 sec)

mysql>
mysql> -- Display index
Query OK, 0 rows affected (0.002 sec)

mysql> SHOW INDEX FROM Sparse_Student;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_co |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| sparse_student | 0 | PRIMARY | 1 | ID | A | 8 | NULL | NULL | | BTREE | | |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.002 sec)

```

Result

The sorted file and indexed retrieval were successfully demonstrated. A true sparse index is generally managed internally by the DBMS rather than manually created in standard SQL.

10. B+ Tree Range Query

Aim

To construct an indexed table and perform a **range query for keys between 15 and 45**.

Procedure

1. Create a table with integer keys.
2. Insert the required records.
3. Create an index on the key.
4. Execute a range query using BETWEEN.
5. Use EXPLAIN to verify index-based access.

```
mysql>
mysql> -- Range query
Query OK, 0 rows affected (0.004 sec)
```

```
mysql> SELECT *
      -> FROM Range_Data
      -> WHERE ID BETWEEN 15 AND 45;
```

```
+-----+
| ID | Name |
+-----+
| 15 | Kavin |
| 20 | Divya |
| 25 | Rahul |
| 30 | Anu   |
| 35 | Vijay |
| 40 | Meena |
| 45 | Ravi  |
+-----+
7 rows in set (0.006 sec)
```

```
mysql>
mysql> -- Check execution plan
Query OK, 0 rows affected (0.004 sec)
```

```
mysql> EXPLAIN
      -> SELECT *
      -> FROM Range_Data
      -> WHERE ID BETWEEN 15 AND 45;
```

```
+-----+
| EXPLAIN
|
+-----+
| -> Filter: (range_data.ID between 15 and 45) (cost=1.66 rows=7)
|   -> Index range scan on Range_Data using PRIMARY over (15 <= ID <= 45) (cost=1.66 rows=7)
|
+-----+
1 row in set (0.010 sec)
```

Result

The B+ Tree-style indexed table was successfully used to retrieve all records with keys between 15 and 45.